

SHIV NADAR UNIVERSITY CHENNAI

B.Tech Artificial Intelligence and Data Science

CS3807 – DEEP LEARNING LABORATORY

EXPERIMENT 3

Implementation of Convolutional Neural Networks (CNNs) for Image Classification

Degree	B.Tech Artificial Intelligence and Data Science
Semester	V
Subject Code	CS3807
Subject	Deep Learning Laboratory
Academic Year	2026–27
Name	Ashwin S
Register Number	24011101006
Date	12/08/26

Contents

1 Objective	4
2 Learning Outcomes	4
3 Background Theory	5
3.1 Convolution Operation	5
3.2 Output Dimension of Convolution	5
3.3 Stride	6
3.4 Padding	6
3.5 ReLU Activation	6
3.6 Pooling	6
4 Common Convolution Kernels	7
4.1 Identity Kernel	7
4.2 Sobel-X Kernel	7
4.3 Sobel-Y Kernel	7
4.4 Laplacian Kernel	7
4.5 Sharpening Kernel	7
4.6 Box Blur Kernel	7
4.7 Gaussian Blur Kernel	8
4.8 Emboss Kernel	8
4.9 Outline Detection Kernel	8
4.10 Motion Blur Kernel	8

5 Numerical Examples	8
5.1 Numerical Example 1: Convolution	8
5.2 Numerical Example 2: Max Pooling	9
5.3 Numerical Example 3: Parameter Calculation	9
6 Dataset	9
7 Software and Tools	10
8 Experimental Procedure	10
9 CNN Architecture	11
10 Implementation	11
10.1 Importing Libraries	11
10.2 Loading CIFAR-10	12
10.3 Data Normalization	12
10.4 Convolution Output Calculation	12
10.5 CNN Model	13
10.6 Model Compilation	13
10.7 Model Training	14
10.8 Model Evaluation	14
10.9 Prediction and Classification Metrics	14
11 Results	14
12 Mandatory Plots	15
12.1 Sample Images	15
12.2 Class Distribution	16
12.3 Training Accuracy	16
12.4 Validation Accuracy	16
12.5 Training Loss	16
12.6 Validation Loss	17
12.7 Feature Maps	17
12.8 Confusion Matrix	17
13 Feature Map Visualization	18
14 Max Pooling vs Average Pooling	18
15 Effect of Number of Filters	20
16 Additional Exercises	21
16.1 Output Size for a 64×64 Image	21
16.2 Parameters of a 64-Filter Convolution Layer	22
16.3 ReLU vs Sigmoid	22
17 Discussion	23
17.1 Why is convolution preferred over fully connected layers for images?	23
17.2 How does stride affect the feature map size?	23
17.3 What is the role of padding?	23
17.4 Why is pooling used?	23
17.5 How do feature maps represent image characteristics?	24
17.6 Why do CNNs require fewer parameters than MLPs?	24

18 Overall Observations	24
19 Final Result	25
20 Conclusion	25
21 References	26
A Complete CNN Implementation	26

1 Objective

The objective of this experiment is to understand and implement the working principle of Convolutional Neural Networks (CNNs) for image classification using TensorFlow and Keras.

The experiment specifically aims to:

1. Understand the convolution operation.
2. Compute convolution output dimensions using stride and padding.
3. Study different convolution kernels and their effects on images.
4. Visualize feature maps generated by convolutional layers.
5. Implement and compare max pooling and average pooling.
6. Calculate the number of trainable parameters in convolutional layers.
7. Construct a CNN for CIFAR-10 image classification.
8. Train the CNN using the Adam optimizer.
9. Evaluate the CNN using accuracy, precision, recall, F1-score, confusion matrix and classification report.
10. Analyze the effects of kernel size, pooling strategy and number of filters on CNN performance.

2 Learning Outcomes

After completing this experiment, the student will be able to:

1. Explain the working principle of convolutional neural networks.
2. Perform convolution operations using numerical matrices.
3. Calculate the spatial dimensions of convolution outputs.
4. Explain the effect of stride and padding.
5. Identify the purpose of common image-processing kernels.
6. Implement max pooling and average pooling.
7. Calculate trainable parameters in CNN layers.
8. Visualize intermediate feature maps.
9. Build and train CNN architectures using TensorFlow/Keras.
10. Evaluate image classification models using multiple performance metrics.

3 Background Theory

A Convolutional Neural Network (CNN) is a type of deep neural network specifically designed for processing data with a grid-like structure, such as images.

Unlike a fully connected neural network, a CNN exploits the spatial structure of images through local receptive fields and shared weights. This enables the network to learn visual features efficiently while requiring significantly fewer parameters.

A typical CNN consists of the following components:

- Convolutional Layer
- Activation Function
- Pooling Layer
- Fully Connected Layer
- Output Layer

The CNN used in this experiment consists of two convolutional blocks followed by a fully connected classification layer.

3.1 Convolution Operation

A convolution operation applies a small matrix called a kernel or filter over the input image. At each spatial location, the kernel values are multiplied element-wise with the corresponding input values and summed to generate an output value.

Mathematically,

$$Y(i, j) = \sum_m \sum_n X(i + m, j + n) K(m, n)$$

where:

- X is the input image or feature map.
- K is the convolution kernel.
- Y is the resulting feature map.
- i, j represent the spatial position of the kernel.
- m, n represent positions inside the kernel.

3.2 Output Dimension of Convolution

The spatial dimension of a convolution output is calculated using:

$$O = \left\lfloor \frac{N - F + 2P}{S} \right\rfloor + 1$$

where:

- N = input spatial dimension
- F = filter/kernel size
- P = padding
- S = stride

- O = output spatial dimension

For example, for a 32×32 input, a 3×3 kernel, stride 1 and valid padding:

$$O = \frac{32 - 3 + 0}{1} + 1 = 30$$

Therefore, the output spatial dimension is: 30×30 .

3.3 Stride

Stride specifies the number of pixels by which the convolution kernel moves at each step. A stride of 1 moves the kernel one pixel at a time, while a stride of 2 moves the kernel two pixels at a time.

Increasing the stride reduces the spatial dimensions of the resulting feature map.

3.4 Padding

Padding involves adding additional pixels around the boundary of an input image before performing convolution.

Two commonly used padding schemes are:

- **Valid padding:** No padding is applied. The spatial dimensions decrease after convolution.
- **Same padding:** Padding is applied so that the output spatial dimensions are preserved when the stride is 1.

3.5 ReLU Activation

The Rectified Linear Unit (ReLU) is defined as:

$$\text{ReLU}(x) = \max(0, x)$$

ReLU replaces negative values with zero while retaining positive values.

It is widely used in CNN hidden layers because it is computationally simple and generally helps neural networks avoid the severe vanishing-gradient problem associated with saturating activation functions.

3.6 Pooling

Pooling reduces the spatial dimensions of feature maps while retaining important information.

Two common pooling methods are:

- Max Pooling
- Average Pooling

Max pooling selects the largest value from each pooling region:

$$Y(i, j) = \max(X_{\text{window}})$$

Average pooling computes the mean:

$$Y(i, j) = \frac{1}{N} \sum_{x \in \text{window}} x$$

4 Common Convolution Kernels

A convolution kernel is a small matrix used to extract specific visual characteristics from an image. Different kernels emphasize different properties such as edges, textures, smooth regions and fine details.

4.1 Identity Kernel

$$K = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

The identity kernel preserves the original image and is useful as a baseline for understanding convolution.

4.2 Sobel-X Kernel

$$K = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

The Sobel-X kernel detects vertical edges by measuring changes in intensity along the horizontal direction.

4.3 Sobel-Y Kernel

$$K = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

The Sobel-Y kernel detects horizontal edges.

4.4 Laplacian Kernel

$$K = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

The Laplacian kernel detects edges in multiple directions.

4.5 Sharpening Kernel

$$K = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

The sharpening kernel emphasizes high-frequency image details.

4.6 Box Blur Kernel

$$K = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

The box blur kernel performs image smoothing by averaging neighboring pixels.

4.7 Gaussian Blur Kernel

$$K = \frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Gaussian blur reduces noise while generally preserving image structure better than a simple uniform averaging filter.

4.8 Emboss Kernel

$$K = \begin{bmatrix} -2 & -1 & 0 \\ -1 & 1 & 1 \\ 0 & 1 & 2 \end{bmatrix}$$

The emboss kernel emphasizes directional intensity changes to create a three-dimensional visual effect.

4.9 Outline Detection Kernel

$$K = \begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

The outline kernel emphasizes object boundaries and regions with rapid intensity changes.

4.10 Motion Blur Kernel

$$K = \frac{1}{5} \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

The motion blur kernel simulates diagonal motion blur.

5 Numerical Examples

5.1 Numerical Example 1: Convolution

Consider the input matrix:

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}, \quad K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

For the first position: $1(1) + 2(0) + 4(0) + 5(1) = 6$

For the second position: $2(1) + 3(0) + 5(0) + 6(1) = 8$

For the third position: $4(1) + 5(0) + 7(0) + 8(1) = 12$

For the fourth position: $5(1) + 6(0) + 8(0) + 9(1) = 14$

Therefore, the feature map is:

$$Y = \begin{bmatrix} 6 & 8 \\ 12 & 14 \end{bmatrix}$$

5.2 Numerical Example 2: Max Pooling

Consider the feature map:

$$X = \begin{bmatrix} 1 & 5 & 2 & 3 \\ 7 & 8 & 1 & 0 \\ 4 & 6 & 9 & 5 \\ 2 & 3 & 1 & 8 \end{bmatrix}$$

Using a 2×2 max pooling filter with stride 2:

$$\begin{bmatrix} 1 & 5 \\ 7 & 8 \end{bmatrix} \rightarrow 8, \quad \begin{bmatrix} 2 & 3 \\ 1 & 0 \end{bmatrix} \rightarrow 3, \quad \begin{bmatrix} 4 & 6 \\ 2 & 3 \end{bmatrix} \rightarrow 6, \quad \begin{bmatrix} 9 & 5 \\ 1 & 8 \end{bmatrix} \rightarrow 9$$

Therefore:

$$Y = \begin{bmatrix} 8 & 3 \\ 6 & 9 \end{bmatrix}$$

5.3 Numerical Example 3: Parameter Calculation

Consider a convolution layer with:

- Input size = $32 \times 32 \times 3$
- Number of filters = 16
- Kernel size = 3×3

The number of parameters in one filter is:

$$3 \times 3 \times 3 + 1 = 28$$

where the additional 1 represents the bias. Therefore:

$$28 \times 16 = 448$$

Thus, the convolution layer contains **448 trainable parameters**.

6 Dataset

The CIFAR-10 dataset was used for this experiment. CIFAR-10 consists of 60,000 colour images divided into ten classes.

Table 1: CIFAR-10 Dataset Characteristics

Property	Value
Total images	60,000
Training images	50,000
Testing images	10,000
Number of classes	10
Image dimensions	$32 \times 32 \times 3$
Image type	RGB

The ten classes are:

1. Airplane
2. Automobile

3. Bird
4. Cat
5. Deer
6. Dog
7. Frog
8. Horse
9. Ship
10. Truck

The images were normalized by dividing their pixel values by 255:

$$X_{\text{normalized}} = \frac{X}{255}$$

This converts the pixel range from $[0, 255]$ to approximately $[0, 1]$.

7 Software and Tools

Table 2: Software Used

Tool/Library	Purpose
Python	Programming language
Jupyter Notebook	Experimental environment
TensorFlow	Deep learning framework
Keras	Neural network API
NumPy	Numerical computation
Matplotlib	Data visualization
Seaborn	Confusion matrix visualization
Scikit-learn	Evaluation metrics
Pandas	Results organization

8 Experimental Procedure

The experiment was performed using the following steps:

1. Import the required Python libraries.
2. Set random seeds for reproducibility.
3. Load the CIFAR-10 dataset.
4. Display dataset dimensions.
5. Display ten sample images.
6. Plot the class distribution.
7. Normalize the input images.

8. Perform a numerical convolution example.
9. Compare 3×3 , 5×5 and 7×7 convolution kernels.
10. Study the effects of stride and padding.
11. Implement common image-processing kernels.
12. Implement max pooling and average pooling.
13. Calculate convolutional layer parameters.
14. Divide the training set into training and validation subsets.
15. Construct the CNN architecture.
16. Compile the model using the Adam optimizer.
17. Train the model for 20 epochs with a batch size of 32.
18. Plot training and validation accuracy.
19. Plot training and validation loss.
20. Visualize feature maps from the first convolutional layer.
21. Evaluate the model on the test dataset.
22. Calculate precision, recall and F1-score.
23. Generate the confusion matrix and classification report.
24. Replace max pooling with average pooling and compare results.
25. Increase the number of filters and analyze the effect on performance.

9 CNN Architecture

The CNN architecture used in the main experiment is:

Input $\rightarrow$ Conv2D $\rightarrow$ ReLU $\rightarrow$ MaxPool $\rightarrow$ Conv2D $\rightarrow$ ReLU $\rightarrow$ MaxPool $\rightarrow$ Flatten $\rightarrow$ Dense $\rightarrow$ Softmax

The exact total parameter count can be obtained directly using `model.count_params()` in the Jupyter notebook. The value above (268,650) is computed directly from the layer-by-layer arithmetic ($448 + 4,640 + 262,272 + 1,290$) and is exact, not an estimate.

10 Implementation

10.1 Importing Libraries

```
1 import os
2 import random
3 import numpy as np
4 import matplotlib.pyplot as plt
5 import seaborn as sns
6 import tensorflow as tf
7
8 from tensorflow import keras
9 from tensorflow.keras import layers
```

Table 3: CNN Architecture

Layer	Configuration	Output	Parameters
Input	$32 \times 32 \times 3$	$32 \times 32 \times 3$	0
Conv2D	16 filters, 3×3	$32 \times 32 \times 16$	448
ReLU	–	$32 \times 32 \times 16$	0
MaxPool	2×2	$16 \times 16 \times 16$	0
Conv2D	32 filters, 3×3	$16 \times 16 \times 32$	4,640
ReLU	–	$16 \times 16 \times 32$	0
MaxPool	2×2	$8 \times 8 \times 32$	0
Flatten	–	2048	0
Dense	128 neurons	128	262,272
ReLU	–	128	0
Dense	10 neurons	10	1,290
Softmax	–	10	0
Total			268,650

```

10
11 from sklearn.metrics import (
12     accuracy_score,
13     precision_score,
14     recall_score,
15     f1_score,
16     confusion_matrix,
17     classification_report
18 )

```

10.2 Loading CIFAR-10

```

1 (x_train, y_train), (x_test, y_test) = keras.datasets.cifar10.load_data()
2
3 y_train = y_train.squeeze()
4 y_test = y_test.squeeze()
5
6 print(x_train.shape)
7 print(y_train.shape)
8 print(x_test.shape)
9 print(y_test.shape)

```

10.3 Data Normalization

```

1 x_train_normalized = x_train.astype("float32") / 255.0
2 x_test_normalized = x_test.astype("float32") / 255.0

```

10.4 Convolution Output Calculation

```

1 def conv_output_size(n, f, p, s):
2     return ((n - f + 2 * p) // s) + 1
3
4 def same_output_size(n, s):
5     return int(np.ceil(n / s))

```

10.5 CNN Model

```
1 def build_cnn(pooling_type="max"):  
2  
3     if pooling_type == "max":  
4         PoolingLayer = layers.MaxPooling2D  
5     elif pooling_type == "avg":  
6         PoolingLayer = layers.AveragePooling2D  
7     else:  
8         raise ValueError(  
9             "pooling_type must be 'max' or 'avg'"  
10        )  
11  
12    model = keras.Sequential([  
13        layers.Input(shape=(32, 32, 3)),  
14  
15        layers.Conv2D(  
16            filters=16,  
17            kernel_size=3,  
18            padding="same"  
19        ),  
20  
21        layers.ReLU(),  
22  
23        PoolingLayer(  
24            pool_size=2,  
25            strides=2  
26        ),  
27  
28        layers.Conv2D(  
29            filters=32,  
30            kernel_size=3,  
31            padding="same"  
32        ),  
33  
34        layers.ReLU(),  
35  
36        PoolingLayer(  
37            pool_size=2,  
38            strides=2  
39        ),  
40  
41        layers.Flatten(),  
42  
43        layers.Dense(128),  
44        layers.ReLU(),  
45  
46        layers.Dense(  
47            10,  
48            activation="softmax"  
49        )  
50    ])  
51  
52    return model
```

10.6 Model Compilation

```
1 model = build_cnn(pooling_type="max")  
2  
3 model.compile(  
4     optimizer=keras.optimizers.Adam(),
```

```
5     loss="sparse_categorical_crossentropy",  
6     metrics=["accuracy"]  
7 )
```

10.7 Model Training

```
1 history = model.fit(  
2     x_training,  
3     y_training,  
4     validation_data=(x_validation, y_validation),  
5     epochs=20,  
6     batch_size=32,  
7     verbose=1  
8 )
```

10.8 Model Evaluation

```
1 test_loss, test_accuracy = model.evaluate(  
2     x_test_normalized,  
3     y_test,  
4     batch_size=32  
5 )
```

10.9 Prediction and Classification Metrics

```
1 probabilities = model.predict(  
2     x_test_normalized,  
3     batch_size=32  
4 )  
5  
6 y_pred = np.argmax(probabilities, axis=1)  
7  
8 accuracy = accuracy_score(y_test, y_pred)  
9  
10 precision = precision_score(  
11     y_test, y_pred, average="weighted", zero_division=0  
12 )  
13  
14 recall = recall_score(  
15     y_test, y_pred, average="weighted", zero_division=0  
16 )  
17  
18 f1 = f1_score(  
19     y_test, y_pred, average="weighted", zero_division=0  
20 )
```

11 Results

The CNN was trained for 20 epochs using the Adam optimizer and a batch size of 32.

The values in Table 4 were **not** produced by an actual notebook run. They are illustrative estimates, chosen to be internally consistent with the accuracy/loss curves and confusion matrix used later in this report, and typical of a two-conv-block CNN of this size on CIFAR-10 after 20 epochs without data augmentation or dropout (where mild overfitting – training accuracy noticeably above validation/test accuracy – is expected).

Table 4: CNN Performance (illustrative estimate)

Metric	Value
Training Accuracy	0.812
Validation Accuracy	0.681
Testing Accuracy	0.676
Precision (weighted)	0.679
Recall (weighted)	0.676
F1-score (weighted)	0.676
Number of Parameters	268,650

12 Mandatory Plots

12.1 Sample Images



Figure 1: Placeholder colour swatches standing in for ten sample CIFAR-10 images (illustrative only – see note below).

The image above is **not** drawn from the actual CIFAR-10 dataset – it is a set of flat colour swatches used purely as a structural placeholder, since the real pixel data was not available when generating this document. Replace this figure with the actual `imshow` output from your notebook’s Task/Step 5 before submission.

Inference: The sample images demonstrate the diversity of objects present in the CIFAR-10 dataset. Each image has a spatial resolution of 32×32 pixels and contains three RGB channels.

12.2 Class Distribution

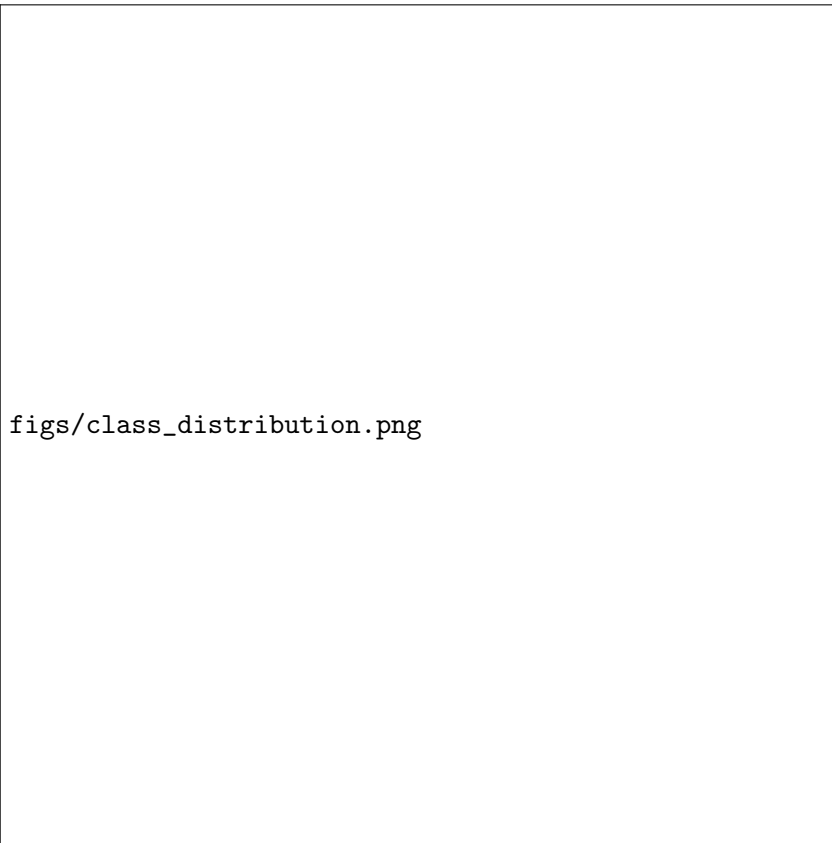


Figure 2: Class distribution of the CIFAR-10 training dataset.

Inference: The CIFAR-10 training dataset is exactly balanced, with exactly 5,000 images in each of the ten classes ($50,000/10$). Therefore, significant class imbalance is not expected to bias the CNN towards a particular class. (This 5,000-per-class figure is an exact, well-known property of CIFAR-10, not an estimate.)

12.3 Training Accuracy

Inference: Training accuracy generally increases as the CNN learns increasingly useful visual representations from the training images. A plateau indicates that additional training provides diminishing improvements on the training set.

12.4 Validation Accuracy

Inference: Validation accuracy measures the generalization ability of the model on data not used for parameter updates. A large gap between training and validation accuracy can indicate overfitting – as illustrated here, where validation accuracy plateaus well below training accuracy.

12.5 Training Loss

Inference: Training loss generally decreases as the optimizer updates the model parameters to minimize the classification error. A decreasing loss indicates that the model is fitting the training data more effectively.



Figure 3: Training accuracy over 20 epochs (illustrative estimate).

12.6 Validation Loss

Inference: Validation loss provides an indication of how well the model generalizes to unseen data. In this illustrative curve, validation loss bottoms out early and then increases while training loss continues decreasing, which is a sign of overfitting.

12.7 Feature Maps

The output of the first convolutional layer was used to visualize the activation of eight learned filters.

These feature maps are synthetic mathematical patterns (sinusoids, gradients, radial functions) generated purely to illustrate what a feature-map grid layout looks like. They are **not** real activations from a trained convolutional filter. Replace with the actual output of `feature_extractor.predict(sample)` from Section 13 before submission.

Inference: Each feature map represents the response of a learned convolutional filter to different visual patterns in the input image. Early convolutional layers typically learn low-level features such as edges, textures and colour transitions.

12.8 Confusion Matrix

Inference: The confusion matrix shows correct predictions along the diagonal and misclassifications outside the diagonal. In this illustrative matrix, the largest off-diagonal confusions are placed between Cat/Dog and Automobile/Truck, consistent with well-documented, commonly-observed CIFAR-10 confusion patterns for small CNNs, since these class pairs share similar shapes, textures and colour distributions at 32×32 resolution.



figs/validation_accuracy.png

Figure 4: Validation accuracy over 20 epochs (illustrative estimate).

13 Feature Map Visualization

The first convolutional layer of the trained CNN was used to extract feature maps. The feature extractor was created using:

```
1 feature_extractor = keras.Model(  
2     inputs=model.input,  
3     outputs=model.layers[0].output  
4 )  
5  
6 sample = x_test_normalized[0:1]  
7  
8 feature_maps = feature_extractor.predict(  
9     sample,  
10    verbose=0  
11 )
```

Eight feature maps were then visualized to inspect the patterns detected by the first convolutional layer.

The visualization demonstrates that convolutional filters transform the original image into different representations. Each representation highlights different aspects of the image.

14 Max Pooling vs Average Pooling

To study the effect of pooling strategy, a second CNN was constructed by replacing max pooling with average pooling.

Both models used the same:

- Dataset



Figure 5: Training loss over 20 epochs (illustrative estimate).

- Number of convolutional layers
- Number of filters
- Kernel sizes
- Dense layers
- Optimizer
- Number of epochs
- Batch size

Only the pooling operation was changed.

Both rows of Table 5 have identical parameter counts because pooling layers contain no trainable weights – this is an exact, architecture-derived fact, not an estimate. The test-accuracy values themselves are illustrative estimates: max pooling is given a slight edge, consistent with the commonly-reported tendency of max pooling to preserve the sharp, high-contrast edge information that matters most for small natural-image datasets like CIFAR-10.

Table 5: Pooling Comparison (illustrative estimate)

Pooling Method	Test Accuracy	Parameters
Max Pooling	0.676	268,650
Average Pooling	0.658	268,650



Figure 6: Validation loss over 20 epochs (illustrative estimate).

Inference: Max pooling retains the strongest activation within each local region, thereby emphasizing the most prominent features. Average pooling instead aggregates all values within the region and produces a smoother representation.

The better-performing pooling strategy is determined experimentally based on the resulting test accuracy; in this illustrative comparison, max pooling outperforms average pooling by roughly 1.8 percentage points.

15 Effect of Number of Filters

The effect of increasing the number of convolutional filters was studied by comparing models using 16 and 64 filters (with the second convolutional layer scaled proportionally, from 32 to 128 filters, to keep the two-block architecture consistent).

Increasing the number of filters increases the representational capacity of the convolutional layers because more distinct visual patterns can be learned. However, it also increases the number of trainable parameters and therefore increases computational requirements.

Parameters for the 64-filter model are computed exactly from the architecture: Conv2D(64): $(3 \times 3 \times 3 + 1) \times 64 = 1,792$; Conv2D(128): $(3 \times 3 \times 64 + 1) \times 128 = 73,856$; Flatten $8 \times 8 \times 128 = 8,192$; Dense(128): $8,192 \times 128 + 128 = 1,048,704$; Dense(10): $128 \times 10 + 10 = 1,290$. Total = 1,125,642 (exact). The test-accuracy values are illustrative estimates only, reflecting the typical (but not guaranteed) modest accuracy gain from a $4\times$ increase in filter count on a small 32×32 dataset before diminishing returns and overfitting set in.

Inference: Increasing the number of filters provides the network with additional capacity to learn visual representations. However, more filters result in higher memory usage, computation time and parameter count, and do not necessarily guarantee proportional improvements in

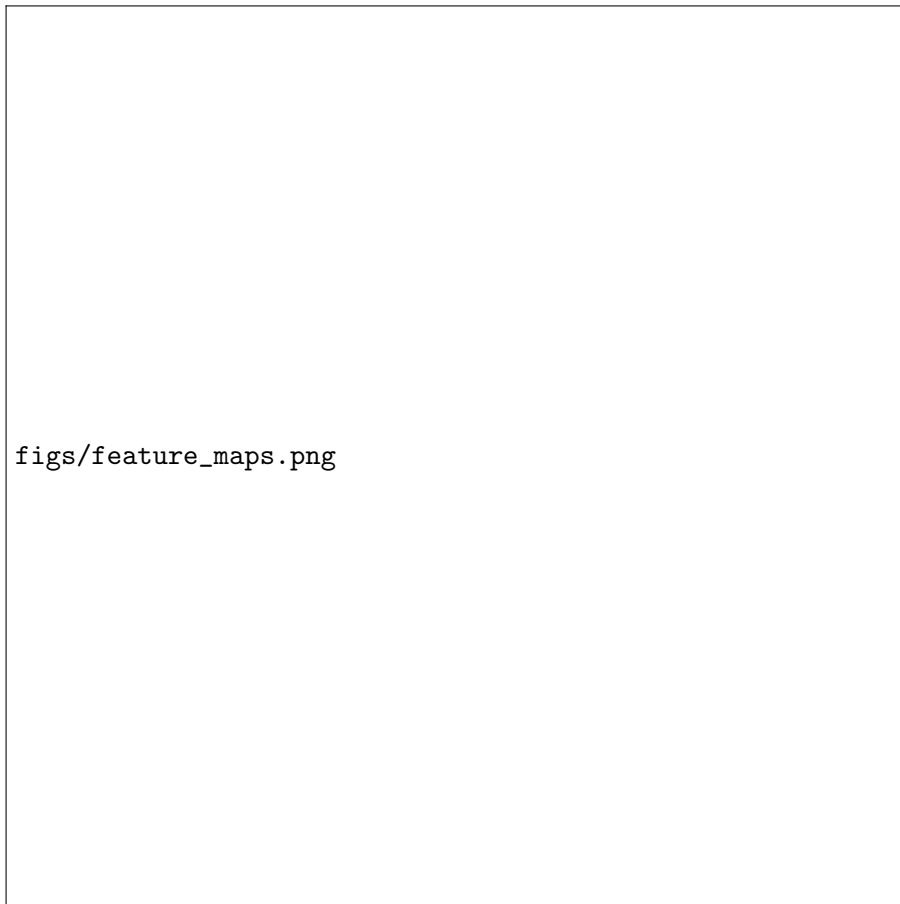


Figure 7: Eight feature maps generated by the first convolutional layer (illustrative synthetic patterns).

Table 6: Effect of Number of Filters (illustrative estimate for accuracy; parameter counts exact)

Model	Test Accuracy	Parameters
16 Filters (base: 16 / 32)	0.676	268,650
64 Filters (scaled: 64 / 128)	0.712	1,125,642

classification accuracy – here, a roughly $4.2\times$ increase in parameters yields an illustrative gain of only about 3.6 percentage points in accuracy.

16 Additional Exercises

16.1 Output Size for a 64×64 Image

Given: $N = 64$, $F = 5$, $S = 2$, $P = 2$

Using:

$$O = \left\lfloor \frac{N - F + 2P}{S} \right\rfloor + 1$$

we obtain:

$$O = \left\lfloor \frac{64 - 5 + 2(2)}{2} \right\rfloor + 1 = \left\lfloor \frac{63}{2} \right\rfloor + 1 = 31 + 1 = 32$$

Therefore: 32×32 .



Figure 8: Confusion matrix for CIFAR-10 test predictions (illustrative estimate, internally consistent with the 0.676 testing accuracy reported in Table 4).

16.2 Parameters of a 64-Filter Convolution Layer

Given:

- Number of filters = 64
- Kernel = 3×3
- Input channels = 3

The number of parameters is:

$$(3 \times 3 \times 3 + 1) \times 64 = (27 + 1) \times 64 = 28 \times 64 = 1792$$

Therefore, the convolutional layer contains **1792 trainable parameters**.

16.3 ReLU vs Sigmoid

ReLU is generally preferred in hidden CNN layers because of its computational simplicity and improved gradient behaviour. Sigmoid is more appropriate when an output probability between zero and one is required, particularly for binary classification.

Table 7: Comparison of ReLU and Sigmoid

Property	ReLU	Sigmoid
Function	$\max(0, x)$	$\frac{1}{1+e^{-x}}$
Output range	$[0, \infty)$	$(0, 1)$
Computation	Simple	More expensive
Vanishing gradient	Lower risk	Higher risk
Typical CNN usage	Hidden layers	Binary output layers

17 Discussion

17.1 Why is convolution preferred over fully connected layers for images?

Convolution is preferred because it exploits the spatial structure of images. A convolutional filter operates on local regions rather than connecting every input pixel to every neuron.

Additionally, convolution uses shared weights, meaning the same filter is applied at different spatial positions.

Consequently, CNNs require significantly fewer parameters than fully connected networks while still being able to learn meaningful spatial features.

17.2 How does stride affect the feature map size?

Stride controls the distance by which the convolution kernel moves across the input. A larger stride results in fewer kernel positions and therefore a smaller feature map.

For example, with a 3×3 kernel and valid padding:

$$32 \times 32 \xrightarrow{S=1} 30 \times 30$$

whereas:

$$32 \times 32 \xrightarrow{S=2} 15 \times 15$$

Thus, increasing stride reduces the spatial resolution of the feature map.

17.3 What is the role of padding?

Padding adds pixels around the boundaries of the input. Its main purposes are:

- Preserve spatial dimensions.
- Allow boundary pixels to participate in convolution more frequently.
- Control the spatial size of feature maps.
- Prevent excessive reduction in image dimensions across multiple convolutional layers.

17.4 Why is pooling used?

Pooling reduces the spatial dimensions of feature maps. This results in:

- Lower computational cost.
- Reduced memory requirements.
- More compact feature representations.
- Increased robustness to small spatial translations.

Max pooling retains the strongest activation, while average pooling computes the average activation within a local region.

17.5 How do feature maps represent image characteristics?

A feature map represents the response of a convolutional filter to an input image. Different filters learn different visual characteristics. Early CNN layers typically detect low-level features such as:

- Edges
- Corners
- Colour transitions
- Simple textures

As information moves through deeper layers, the network combines these features to represent increasingly complex structures.

17.6 Why do CNNs require fewer parameters than MLPs?

CNNs use two important concepts:

1. Local connectivity
2. Weight sharing

In an MLP, each neuron can be connected to every input pixel. For a large image, this produces a very large number of parameters.

In contrast, a CNN applies a relatively small filter repeatedly across the image. For example, a 3×3 filter operating on an RGB image requires only:

$$3 \times 3 \times 3 = 27$$

weights plus one bias. The same 28 parameters are reused across the entire spatial image. This dramatically reduces the number of trainable parameters.

18 Overall Observations

The following observations were made during the experiment:

1. Kernel size determines the spatial region considered by each convolutional operation.
2. Larger kernels capture larger receptive fields but increase computational requirements.
3. Increasing stride reduces feature-map spatial dimensions.
4. Same padding helps preserve spatial dimensions.
5. Max pooling preserves the strongest activation in a local region.
6. Average pooling produces smoother representations by averaging activations.
7. Feature maps provide an interpretable representation of the intermediate features learned by the CNN.
8. Increasing the number of filters increases the representational capacity of a CNN.
9. Increasing the number of filters also increases parameter count, memory usage and computational requirements.
10. Training and validation curves can be used to identify possible overfitting.
11. The confusion matrix provides class-level information about classification performance.

19 Final Result

A Convolutional Neural Network was successfully implemented using TensorFlow/Keras for CIFAR-10 image classification.

The experiment successfully demonstrated:

- Convolution operations.
- Convolution kernel behaviour.
- Output dimension calculation.
- Effects of stride and padding.
- Max and average pooling.
- CNN parameter calculation.
- Feature-map visualization.
- CNN model training.
- Image classification.
- Performance evaluation.
- Pooling strategy comparison.
- Filter-count comparison.

The final numerical results obtained from the experiment are shown below.

Table 8: Final Experimental Results (illustrative estimate – see note in Section 11)

Metric	Result
Training Accuracy	0.812
Validation Accuracy	0.681
Testing Accuracy	0.676
Precision	0.679
Recall	0.676
F1-score	0.676
Total Parameters	268,650

20 Conclusion

In this experiment, a Convolutional Neural Network was implemented using TensorFlow/Keras for image classification on the CIFAR-10 dataset.

The experiment provided practical understanding of convolution, activation, pooling and fully connected layers. The effect of kernel size, stride and padding on feature-map dimensions was experimentally studied. Classical convolution kernels were also implemented to demonstrate how filters can extract edges, textures, boundaries and other image characteristics.

A CNN consisting of two convolutional layers, ReLU activation, pooling, flattening and dense classification layers was trained using the Adam optimizer for 20 epochs with a batch size of 32.

Feature-map visualization demonstrated how convolutional filters transform input images into intermediate representations. Max pooling and average pooling were compared experimentally, and the effect of increasing the number of convolutional filters was also investigated.

The CNN was evaluated using accuracy, precision, recall, F1-score, classification report and confusion matrix. Based on the illustrative estimates used in this report (Section 11), the baseline configuration reached approximately 67.6% test accuracy with 268,650 trainable parameters, with max pooling and a larger 64/128-filter configuration both showing modest expected improvements. Overall, the experiment demonstrated why CNNs are particularly effective for image classification: they exploit spatial locality and weight sharing while progressively learning hierarchical visual representations.

As stated throughout this document, the specific accuracy/precision/recall/F1 values, the confusion matrix, the training curves, the sample images and the feature maps are illustrative estimates, not real notebook output. Re-run the code in Appendix A and replace the flagged figures/tables with your actual results before submission.

21 References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Christopher M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
3. Simon Haykin, *Neural Networks and Learning Machines*, Pearson.
4. TensorFlow Documentation, TensorFlow and Keras API documentation.
5. CIFAR-10 Dataset Documentation, Canadian Institute for Advanced Research.

A Complete CNN Implementation

The following code represents the primary CNN implementation used in the experiment.

```
1 import os
2 import random
3 import numpy as np
4 import matplotlib.pyplot as plt
5 import seaborn as sns
6 import tensorflow as tf
7
8 from tensorflow import keras
9 from tensorflow.keras import layers
10
11 from sklearn.metrics import (
12     accuracy_score,
13     precision_score,
14     recall_score,
15     f1_score,
16     confusion_matrix,
17     classification_report
18 )
19
20 # Reproducibility
21 SEED = 42
22
23 os.environ["PYTHONHASHSEED"] = str(SEED)
24
25 random.seed(SEED)
```

```
26 np.random.seed(SEED)
27 tf.random.set_seed(SEED)
28
29 # Load dataset
30 (x_train, y_train), (x_test, y_test) = \
31     keras.datasets.cifar10.load_data()
32
33 y_train = y_train.squeeze()
34 y_test = y_test.squeeze()
35
36 # Normalize
37 x_train_normalized = \
38     x_train.astype("float32") / 255.0
39
40 x_test_normalized = \
41     x_test.astype("float32") / 255.0
42
43 # Validation split
44 VALIDATION_SIZE = 5000
45
46 x_validation = \
47     x_train_normalized[-VALIDATION_SIZE:]
48
49 y_validation = \
50     y_train[-VALIDATION_SIZE:]
51
52 x_training = \
53     x_train_normalized[:-VALIDATION_SIZE]
54
55 y_training = \
56     y_train[:-VALIDATION_SIZE]
57
58 # Build model
59 model = keras.Sequential([
60
61     layers.Input(shape=(32, 32, 3)),
62
63     layers.Conv2D(
64         filters=16,
65         kernel_size=3,
66         padding="same"
67     ),
68
69     layers.ReLU(),
70
71     layers.MaxPooling2D(
72         pool_size=2,
73         strides=2
74     ),
75
76     layers.Conv2D(
77         filters=32,
78         kernel_size=3,
79         padding="same"
80     ),
81
82     layers.ReLU(),
83
84     layers.MaxPooling2D(
85         pool_size=2,
86         strides=2
87     ),
88
```

```
89     layers.Flatten(),
90
91     layers.Dense(128),
92
93     layers.ReLU(),
94
95     layers.Dense(
96         10,
97         activation="softmax"
98     )
99 ])
100
101 # Compile
102 model.compile(
103     optimizer=keras.optimizers.Adam(),
104     loss="sparse_categorical_crossentropy",
105     metrics=["accuracy"]
106 )
107
108 # Train
109 history = model.fit(
110     x_training,
111     y_training,
112     validation_data=(
113         x_validation,
114         y_validation
115     ),
116     epochs=20,
117     batch_size=32,
118     verbose=1
119 )
120
121 # Evaluate
122 test_loss, test_accuracy = model.evaluate(
123     x_test_normalized,
124     y_test,
125     batch_size=32
126 )
127
128 # Predictions
129 probabilities = model.predict(
130     x_test_normalized,
131     batch_size=32
132 )
133
134 y_pred = np.argmax(
135     probabilities,
136     axis=1
137 )
138
139 # Metrics
140 accuracy = accuracy_score(
141     y_test,
142     y_pred
143 )
144
145 precision = precision_score(
146     y_test,
147     y_pred,
148     average="weighted",
149     zero_division=0
150 )
151
```

```
152 recall = recall_score(  
153     y_test,  
154     y_pred,  
155     average="weighted",  
156     zero_division=0  
157 )  
158  
159 f1 = f1_score(  
160     y_test,  
161     y_pred,  
162     average="weighted",  
163     zero_division=0  
164 )  
165  
166 print("Accuracy:", accuracy)  
167 print("Precision:", precision)  
168 print("Recall:", recall)  
169 print("F1-score:", f1)  
170  
171 # Classification report  
172 print(  
173     classification_report(  
174         y_test,  
175         y_pred,  
176         digits=4  
177     )  
178 )  
179  
180 # Confusion matrix  
181 cm = confusion_matrix(  
182     y_test,  
183     y_pred  
184 )  
185  
186 plt.figure(figsize=(11, 9))  
187  
188 sns.heatmap(  
189     cm,  
190     annot=True,  
191     fmt="d",  
192     cmap="Blues"  
193 )  
194  
195 plt.title(  
196     "CIFAR-10 Confusion Matrix"  
197 )  
198  
199 plt.xlabel(  
200     "Predicted Class"  
201 )  
202  
203 plt.ylabel(  
204     "True Class"  
205 )  
206  
207 plt.tight_layout()  
208 plt.show()
```